

Zweite Runde des 44.
Bundeswettbewerbs Informatik

Aufgabe 2

[Name Redacted]

[School Redacted]

1 Vorstellung der Aufgabe

Die zweite Aufgabe der zweiten Runde des 44. Bundeswettbewerbs Informatik beschreibt ein Szenario, in dem Pflanzen in einem Botanischen Garten automatisiert durch Roboter bewässert werden müssen.

Dafür muss ein Programm entwickelt werden, das für eine vorgegebene Anzahl an Pflanzen mit ebenfalls vorgegebenen Positionen einen Weg findet, alle Pflanzen mit einer möglichst geringen Anzahl an Robotern zu bewässern. Dabei ist die Distanz, die die Roboter zurücklegen, durch eine maximale Reiselänge begrenzt. Die Startpositionen der Roboter dürfen beliebig gewählt werden, diese müssen jedoch am Ende wieder erreicht werden.

Im Folgenden wird davon ausgegangen, dass für die Koordinaten der Pflanzen gilt x und $y \in \mathbb{N}$, sowie x und $y \in \mathbb{R}$ für die selbstgewählte Startpositionen der Roboter.

2 Lösung

Im Folgenden wird zunächst das Problem erfasst und interpretiert, anschließend wird basierend auf den gewonnenen Erkenntnissen ein strukturiertes Lösungsverfahren aufgebaut, welches schließlich auf Grundlage von verschiedenen Testfällen bewertet wird. Abschließend wird die Vorgehensweise bewertet.

2.1 Auffassung des Problems

Damit eine sinnvolle Lösung gefunden werden kann, muss das Problem zunächst analysiert und eingeordnet werden. Hierbei kann häufig ein Bezug zu bereits bekannten Problemstellungen hergestellt werden, was das weitere systematische Vorgehen präzisiert.

2.1.1 Das Capacitated Multiple Travelling Salesmen Problem

Das Capacitated Multiple Travelling Salesmen Problem (CMTSP), oft auch Vehicle Routing Problem genannt, beschreibt einen Sachverhalt, bei dem für eine vorgegebene Anzahl an Handelsreisenden¹ eine Route gefunden werden muss, welche alle vorgegebenen Orte besucht. Dabei darf eine maximale

¹Engl.: „Salesmen“

Reiselänge pro Handelsreisenden nicht überschritten werden. Es ist eine Variation des Multiple Travelling Salesmen Problem, welches wiederum eine Variation des Travelling Salesman Problem (TSP) ist. Ein TSP-Entscheidungsproblem stellt die Frage, ob es einen Rundweg mit einer Routenlänge unter einem bestimmten Wert gibt.²

Das TSP-Entscheidungsproblem ist ein klassisches NP-vollständiges Problem. Das bedeutet, dass sich Lösungen zwar in Polynominalzeit überprüfen, jedoch vermutlich nicht in Polynominalzeit finden lassen. Der Zusatz als vollständiges und schweres NP Problem bedeutet, dass sich jedes Problem in NP in Polynominalzeit darauf reduzieren lässt.³ NP-vollständige Probleme sind daher laufzeittechnisch sehr aufwendig.⁴

Allgemein gilt für das TSP mit einer Anzahl n an zu besuchenden Orten, unter Verwendung eines dynamischen Ansatz, die Laufzeitkomplexität:

$$O(n^2 \cdot 2^n)$$

Für eine Brute Force Lösung gilt sogar:

$$O(n!)$$

Das bedeutet, dass eine exakte Lösung für große n nicht zielführend ist, da die Berechnungsdauer im besten Fall exponentiell wächst.⁵ Steht eine festgelegte Anzahl an Handelsreisenden zur Verfügung, wird die Laufzeitberechnung noch komplexer. Auch der Zusatz der begrenzten maximalen Routenlänge des CMTSP erschwert die Berechnung, da nun ein Großteil der möglichen Routen verworfen werden müssen, um eine passende Anzahl an Handelsreisenden zu finden. Aus diesem Grund kann man annehmen, dass auch die Laufzeitkomplexität für das CMTSP größer als $O(n^2 \cdot 2^n)$ ist.

2.1.2 Einordnung der Aufgabe

Die Aufgabenstellung beschreibt eine Menge an Pflanzen, welche besucht werden müssen. Diese Pflanzen entsprechen den zu besuchenden Punkten im TSP. Zusätzlich sollen mehrere Roboter eingesetzt werden, um die

²Vgl. <https://neos-guide.org/case-studies/tra/multiple-traveling-salesman-problem-mtsp/>

³Vgl. [https://quartz-akz.pages.dev/Travelling-Salesman-Problem-\(TSP\)](https://quartz-akz.pages.dev/Travelling-Salesman-Problem-(TSP))

⁴Unter der Annahme, dass $P \neq NP$

⁵Vgl. <https://antoniocanada.medium.com/introduction-to-the-travelling-salesman-problem-5ace44932cb5>

Pflanzen zu gießen, diese Roboter können als Handelsreisende interpretiert werden. Schließlich muss die Anzahl der Roboter selbst gewählt und so gering wie möglich gehalten werden, basierend auf einer vorgegebenen maximalen Routenlänge. Um so wenige Roboter wie möglich zu verwenden, muss ein Roboter möglichst viele Pflanzen abdecken, was wiederum eine Optimierung der Routen verlangt. Das in der Aufgabenstellung beschriebene Problem lässt sich also als CMTSP auffassen. Damit gilt allerdings auch, dass die Suche nach einer optimalen Lösung, aufgrund der extremen Laufzeitkomplexität und dem potentiellen Vorhandensein von Tausenden Pflanzen, nicht umsetzbar ist.

2.2 Heuristischer Ansatz

Alternativ zum Versuch, eine exakte Lösung zu finden, gibt es im Allgemeinen zwei weitere Verfahren. Neben einer Approximation, welche eine bestimmte Genauigkeit mathematisch garantiert, existieren auch sogenannte heuristische Ansätze. Diese sind Verfahren, welche sich bewährt haben, gute Ergebnisse zu liefern, jedoch keine vorher absehbare Präzision zusichern. Gleichzeitig profitiert diese Vorgehensweise allerdings häufig von einem drastisch verbesserten Laufzeitverhalten.⁶

2.2.1 Konkreter Lösungsansatz

Der folgende Abschnitt befasst sich mit der Umsetzung eines heuristischen Lösungsverfahrens in der Programmiersprache Rust. Zur Darstellung der Positionen der Pflanzen und Roboter wird das folgende Struct verwendet:

```
pub struct Point {  
    pub y: f32,  
    pub x: f32  
}
```

Es verfügt unter Anderem über die folgenden Hilfsfunktionen:

```
pub fn calc_dis(&self, other: &Point) -> f32 {  
    // Euclidean distance  
    ((self.x - other.x).powi(2) + (self.y - other.y).powi(2)).sqrt()  
}  
  
pub fn calc_dis_squared(&self, other: &Point) -> f32 {  
    (self.x - other.x).powi(2) + (self.y - other.y).powi(2)  
}
```

⁶Vgl. <https://www.uni-ulm.de/algo211/>

Als Ausgangspunkt der Lösung dient diese Funktion:

```
pub fn solve(max_path_length: usize, plants: &Vec<Point>)
-> (usize, Vec<Point>, Vec<Vec<Point>> > {
    println!("Allowed max length: {}", max_path_length);
    println!("Number of plants: {}", plants.len());

    let max_considered_k = plants.len();
    let (k, centers, routes) = find_ideal_k(plants, max_considered_k, max_path_length);

    println!("Number of robots: {k}");
    for (i, center) in centers.iter().enumerate() {
        println!("Starting position: X:{}, Y:{}", center.x, center.y);

        // Removing the first and last element, since it is the center itself
        let mut route = routes[i].clone();
        route.remove(0);
        route.remove(route.len() - 1);
        for plant in route {
            println!("Visit plant: X:{}, Y:{}", plant.x, plant.y);
        }
    }
    (k, centers, routes)
}
```

Sie erhält die maximale erlaubte Routenlänge sowie eine Liste der Koordinaten der Pflanzen als Parameter und gibt die Anzahl an benötigten Robotern, deren Position, sowie eine Liste aus Listen mit Punkten, welche die Routen der einzelnen Roboter enthält, zurück. Eine Route beginnt und endet mit der jeweiligen Startposition des Roboters. Die Funktion dient hauptsächlich als Wrapper für die folgende Funktion, um das debuggen zu erleichtern:

```
fn find_ideal_k(data_points: &Vec<Point>, mut max_k: usize, max_path_length: usize)
-> (usize, Vec<Point>, Vec<Vec<Point>> > {
    let mut best_k = max_k;
    let mut best_centers = Vec::new();
    let mut best_routes = Vec::new();
    let mut min_k = 1;

    // Binary searching for k
    while min_k <= max_k {
        let mid_k = (min_k + max_k) / 2;
        println!("... trying k={mid_k}");

        // Running k-Means
        let mut best_wcss = f32::MAX;
        let mut centers = Vec::new();
        let mut assignment = Vec::new();
        // Running multiple times since k-means++ initialization is in part random
    }
```

```

for _ in 0..3 {
  let (cen, wcss, ass) = run_k_means(data_points, mid_k);
  // Only keeping the best run
  if wcss < best_wcss {
    best_wcss = wcss;
    centers = cen;
    assignment = ass;
  }
}

// Creating the route
let result = run_greedy(data_points, &centers, &assignment, max_path_length);
match result {
  // If we get a valid result we will try smaller k
  Some(r) => {
    best_k = mid_k;
    best_centers = centers;
    best_routes = r;

    if mid_k == 0 {
      break;
    }
    max_k = mid_k - 1;
  }
  // If not we will try bigger k
  None => {
    min_k = mid_k + 1;
  }
}

// Returning the lowest valid result
(best_k, best_centers, best_routes)
}

```

Im Kern führt die Funktion den k-Means-Algorithmus mit unterschiedlichen k aus, bis der niedrigst mögliche Wert für k gefunden wurde. Dafür führt die Funktion in der ersten Iteration k-Means mit der Hälfte des Werts von max_k aus. Da die k-Means-Implementierung die Clusterzentren nicht deterministisch initialisiert, wodurch unterschiedlich qualitative Ergebnisse erzielt werden können, wird k-Means mehrmals pro Iteration durchgeführt, wobei nur das beste Ergebnis verwendet wird. Anschließend wird ein Greedyverfahren verwendet, um die Routen basierend auf den berechneten Clusterzentren zu erstellen. Ist keine der Routen länger als die vorgegebene maximale Routenlänge, wird nun nach kleineren validen k gesucht, ansonsten wird nach größeren gesucht. Dabei wird jeweils der Bereich über, beziehungsweise unter, dem aktuellen k betrachtet und der Prozess wiederholt, wodurch

eine effiziente Binärsuche nach dem niedrigsten validen k durchgeführt wird. Sobald $\min_k$ größer ist als $\max_k$ ist der beste Wert erreicht und das vorherige Ergebnis kann zurückgegeben werden. Dies ist die dabei verwendete k-Means-Implementierung:

```
fn run_k_means(data_points: &Vec<Point>, centers_count: usize)
-> (Vec<Point>, f32, Vec<usize>) {
    // Initializing centers using k-means++ approach
    let mut rng = rand::rng();
    let mut centers = Vec::with_capacity(centers_count);
    // First center will be a random data point
    centers.push(data_points.choose(&mut rng).unwrap().clone());
    // Repeating until centers vec contains the required number of centers
    while centers.len() < centers_count {
        let mut distances: Vec<f32> = Vec::new();
        // Computing the minimum squared distance to each center for every data point
        for data_point in data_points.iter() {
            let mut min_distance = f32::MAX;
            for center in centers.iter() {
                let distance = data_point.calc_dis_squared(center);
                if distance < min_distance {
                    min_distance = distance;
                }
            }
            distances.push(min_distance);
        }
        let sum = distances.iter().sum();
        let mut ran = if sum == 0_f32 {0_f32} else {rng.random_range(0.0..sum)};
        // Picking a new center based on the distance to the nearest center
        // using weighted random sampling
        for (i, &distance) in distances.iter().enumerate() {
            ran -= distance;
            if ran <= 0.0 {
                centers.push(data_points[i].clone());
                break;
            }
        }
    }

    let data_points_count = data_points.len();
    let mut total_center_change;

    // Main calculation loop
    loop {
        // Resetting total change after each iteration
        total_center_change = 0_f32;

        // Assigning data points to centers
        let mut assignment = vec![0; data_points_count];
        // Iterating over all data points
```

```

for i in 0..data_points_count {
  let mut min_distance: f32 = f32::MAX;
  // Checking the distance to each center
  for x in 0..centers_count {
    let cur_dis = data_points[i].calc_dis(&centers[x]);
    // Data point will be assigned to the closest center
    if cur_dis < min_distance {
      min_distance = cur_dis;
      assignment[i] = x;
    }
  }
}

// Calculating new centers
let mut sum_x: Vec<f32> = vec![0_f32; centers_count];
let mut sum_y: Vec<f32> = vec![0_f32; centers_count];
let mut count: Vec<i32> = vec![0; centers_count];
// Calculating the sum of all x and y coordinates of all points of each center
for i in 0..data_points_count {
  let ass = assignment[i];
  sum_x[ass] += data_points[i].x;
  sum_y[ass] += data_points[i].y;
  count[ass] += 1;
}

// Iterating over each center to calculate the new ones
for i in 0..centers_count {
  // If no data is being assigned to a center, we will reinitialize it
  if count[i] == 0 {
    centers[i] = data_points.choose(&mut rng).unwrap().clone();
    continue;
  }
  // Otherwise the new position will be the average position of
  // all data points assigned to this center
  let new_point = Point::new(sum_x[i] / count[i] as f32, sum_y[i] / count[i] as f32);
  let old_point = &centers[i];
  total_center_change += (new_point.x - old_point.x).abs();
  total_center_change += (new_point.y - old_point.y).abs();
  centers[i] = new_point;
}

// Checking if the total change is small enough to call it a day
if total_center_change < 1e-2 {
  let mut wcss = 0_f32;
  // Calculating within cluster sum of squares (wcss)
  for i in 0..data_points_count {
    let center = assignment[i];
    wcss += data_points[i].calc_dis_squared(&centers[center]);
  }

  return (centers, wcss, assignment)
}

```



```

    }
  }
}

```

Zu Beginn werden die Clusterzentren mit Hilfe des k-Means++-Verfahrens initialisiert, dabei haben Zentren eine höhere Wahrscheinlichkeit, eine Startposition zugewiesen zu bekommen, die weiter von anderen Zentren entfernt ist.⁷ Dies führt dazu, dass eine gute Anfangsverteilung wahrscheinlich ist. Anschließend wird der eigentliche k-Means-Algorithmus durchgeführt, indem die Datenpunkte wiederholt dem nächsten Zentrum zugewiesen und diese darauf basierend aktualisiert werden. Wenn die Zentren in einer Iteration ihre Position nur noch geringfügig anpassen, werden die Koordinaten der Zentren als vollständig identifiziert angesehen und zusammen mit ihrer Anzahl sowie der Zuweisung der Datenpunkte zu den Zentren zurückgegeben. Ein Greedy-Algorithmus löst ein Problem, indem er die ideale Lösung für jeden Teilschritt eines Problems sucht. In der folgenden Implementierung erzielt er die Routenwahl durch die Wahl des jeweiligen nächsten Datenpunkts zum aktuellen Punkt:

```

fn run_greedy(data_points: &Vec<Point>, centers: &Vec<Point>,
              assignment: &Vec<usize>, max_path_length: usize)
-> Option<Vec<Vec<Point>>> {
  // Assigning each center a list of indices that belong to it
  let mut assigned_point_indices: Vec<Vec<usize>> = vec![Vec::new(); centers.len()];
  for (i, center_index) in assignment.iter().enumerate() {
    assigned_point_indices[*center_index].push(i);
  };

  // This will store a route for each center
  let mut routes: Vec<Vec<Point>> = Vec::with_capacity(centers.len());
  // Now we will fill this list by calculating the ideal path; This is the actual
  // greedy algorithm
  // Iterating over every center to find the best route for each one
  for (i, center) in centers.iter().enumerate() {
    let mut current = *center;
    let mut route = vec![current];
    let mut unvisited = assigned_point_indices[i].clone();

    // Running until no points are left to visit
    while !unvisited.is_empty() {
      let mut min_dis = f32::MAX;
      let mut min_index = 0;
      // Finding the min distance between the current point and all other points
      // in the cluster

```

⁷Vgl. <http://ilpubs.stanford.edu:8090/778/1/2006-13.pdf> (S. 3)

```

    for (x, assigned_index) in unvisited.iter().enumerate() {
        let cur_dis = current.calc_dis_squared(&data_points[*assigned_index]);
        if cur_dis < min_dis {
            min_dis = cur_dis;
            min_index = x;
        }
    }

    let next_point = data_points[unvisited[min_index]].clone();
    current = next_point.clone();
    // Adding it to the route and moving to the next
    route.push(current);
    // Using swap_remove which is O(1) (unlike remove())
    unvisited.swap_remove(min_index);
}

// Adding the center itself to the route, since we want to get back to
// the starting point
route.push(*center);

// Running an optimization algorithm over the route
two_opt(&mut route);

// Finally calculating the total distance
let mut cumulative_distance = 0_f32;
for neighbor_pair in route.windows(2) {
    cumulative_distance += neighbor_pair[0].calc_dis(&neighbor_pair[1]);
}

// Checking if the route is valid, if not we can exit early
if cumulative_distance > max_path_length as f32 {
    return None;
}

// Calculated route will be added to the routes
routes.push(route);
}

// After iterating we return the results
Some(routes)
}

```

Die Route beginnt immer mit dem Zentrum selbst, der nächste Datenpunkt in der Route ist der, welcher am nächsten am Zentrum ist. Daraufhin wird wieder der Datenpunkt mit der geringsten Distanz ermittelt. Dies wird so lange wiederholt, bis alle Datenpunkte besucht sind. Da der Roboter am Ende wieder zum Startpunkt zurückkehren soll, wird zu letzt noch das Zentrum selbst angefügt. Jede Route beginnt und endet also am Startpunkt. Bevor die finale Routenlänge berechnet wird, wird ein Optimierungsverfahren

über die Route gelaufen. Ist die Distanz größer als die maximal erlaubte Distanz, wird sofort abgebrochen, da jede Lösung, in der auch nur ein einziger Roboter die maximale Routenlänge überschreitet, ungültig ist. Ansonsten wird das Verfahren Schritt für Schritt für alle weiteren Zentren wiederholt. Sollten für jedes Zentrum eine valide Route existieren, werden alle Routen zurückgegeben.

Das verwendete Optimierungsverfahren ist das 2-Opt-Verfahren:

```
fn two_opt(route: &mut Vec<Point>) {
    // No optimization needed/possible (basically 3 points since start = end)
    if route.len() < 4 {
        return;
    }

    let mut improved = true;

    while improved {
        improved = false;

        // Running for every Point in the route,
        // besides the last one to make sure that i+1 is valid
        for i in 0..route.len() - 2 {
            // Running for all points ahead, starting with the one after the next
            for j in i + 2..route.len() - 1 {
                // Calculating the current distance between the current two pairs
                let current_dist = route[i].calc_dis_squared(&route[i + 1]) +
                    route[j].calc_dis_squared(&route[j + 1]);
                // Calculating the distance if we switched the connection between the pairs
                let new_dist = route[i].calc_dis_squared(&route[j]) +
                    route[i + 1].calc_dis_squared(&route[j + 1]);

                // If this is an optimization we apply the new route
                if new_dist < current_dist {
                    route[i + 1..=j].reverse();
                    improved = true;
                    // Running again to check if we can apply more optimizations
                    break;
                }
            }
        }

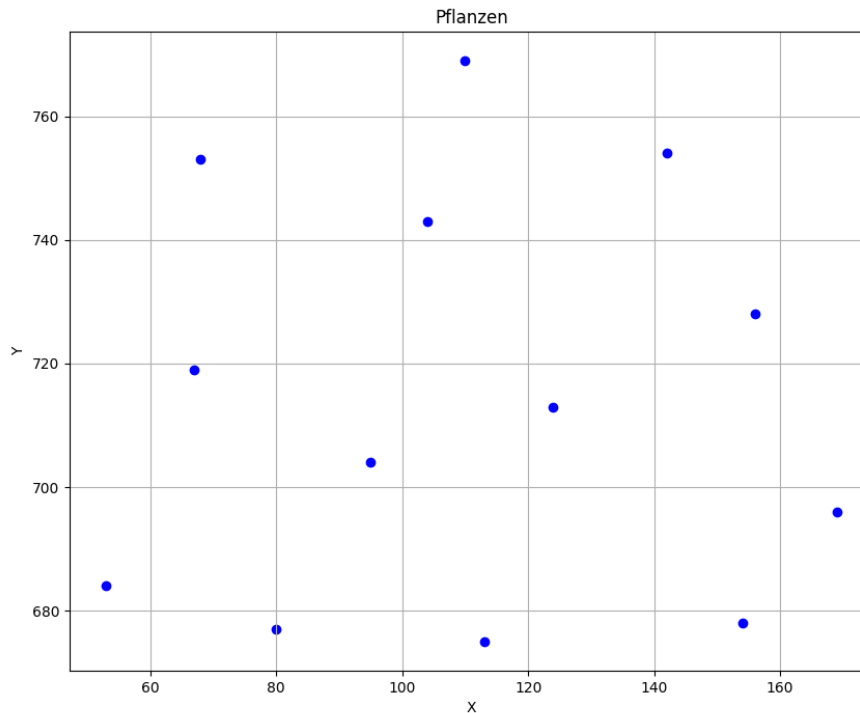
        // Running again to check if we can apply more optimizations
        if improved { break; }
    }
}
```

Durch 2-Opt kann die Routenlänge verbessert werden, indem über die Punkte der Route iteriert wird, dabei wird immer die Routenlänge eines Teilseg-

ments mit einem durch einen Kantentausch der Grenzen des Segments entstandenem, neuem Teilsegment verglichen. Ist die Verbindung kürzer als die aktuelle, dann wird diese übernommen und dessen dazwischenliegender Abschnitt invertiert.⁸

2.2.2 Analyse anhand von Beispielen

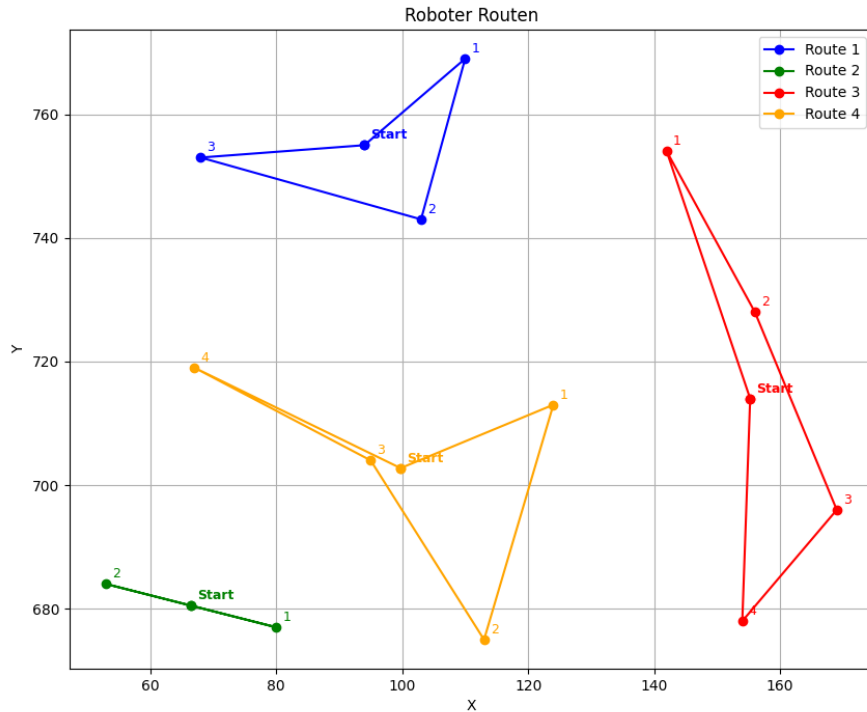
Um zu verstehen, welche Ergebnisse der Algorithmus liefert, wird nun ein Beispiel aus 13 Pflanzen betrachtet.⁹ Die maximale Routenlänge ist 175 und die Pflanzen sind wie folgt verteilt:



Das Programm berechnet dafür die folgende Lösung:

⁸Vgl. <https://www.cs.ubc.ca/~hutter/previous-earg/EmpAlgReadingGroup/TSP-JohMcg97.pdf> (S 16ff.)

⁹Vgl. https://bwinf.de/fileadmin/wettbewerbe/bundeswettbewerb/44/2_Runde/Aufgaben442.pdf (Aufgabe2, roboter01.txt)

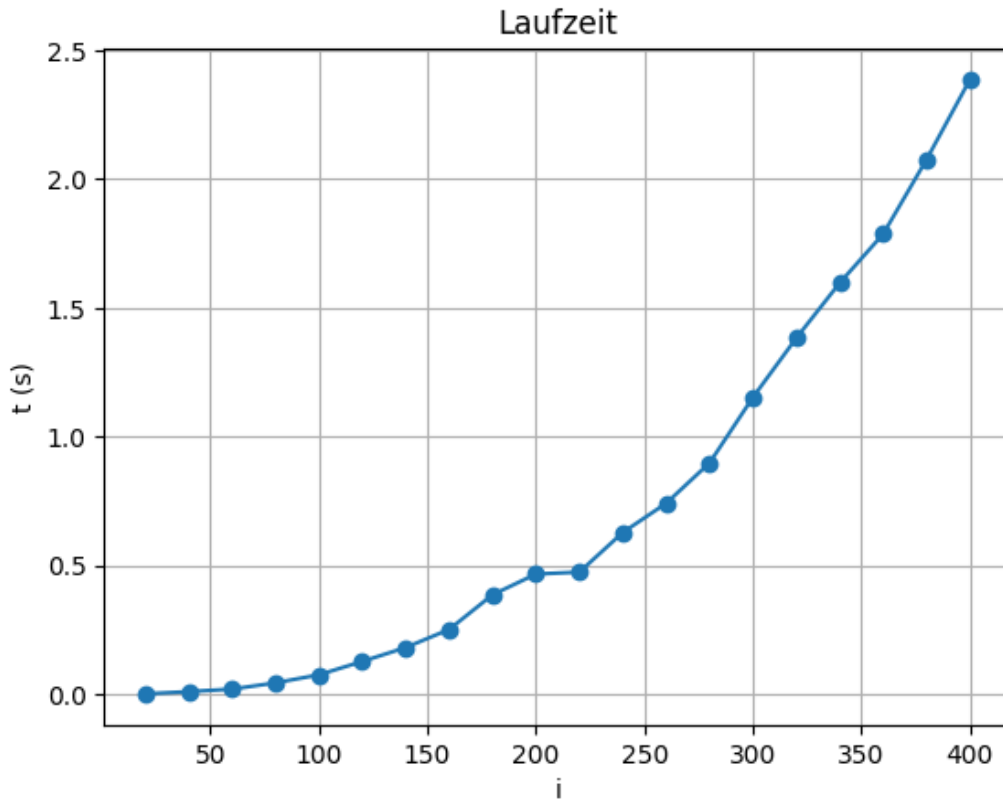


Man erkennt, dass die mit dem Label „Start“ gekennzeichneten Startpunkte der Roboter sinnvoll gewählt wurden. Insgesamt erkennt das Programm vier Roboter als niedrigste mögliche Anzahl, was gleichzeitig die für dieses Beispiel niedrigste mögliche ist.

Für etwa 1000 Pflanzen liefert das Programm, wenn auf moderner Hardware ausgeführt, durchschnittlich in ein bis zwei Minuten ein Ergebnis. Bei 5000 Pflanzen beträgt die Berechnungszeit jedoch schon über eine Stunde.

2.2.3 Bewertung der Vorgehensweise

Allgemein sind die Ergebnisse qualitativ gut. Für kleinere Mengen an Pflanzen ist auch die Berechnungsdauer in Ordnung. Für mehr als 5000 Pflanzen wird die benötigte Zeit jedoch schnell zu groß, um realistisch anwendbar zu sein, da die Berechnungszeit polynomial steigt, wie aus dem folgenden Laufzeitdiagramm hervorgeht:



Zunächst könnte man dies verbessern, indem man Teile des Programms parallel ausführt. Eine einfach umzusetzende, jedoch nicht besonders effektive Multithreading Maßnahme ist es, die verwendete Binärsuche auf mehrere Threads aufzuteilen, indem 2^r Threads jeweils r Stufen voraus rechnen. Alternativ sind auch komplexere Multithreading Implementierungen möglich, welche für eine bessere Leistung bei einer große Anzahl an zur Verfügung stehenden Threads sorgen, jedoch wäre dies deutlich schwieriger zu implementieren als die erwähnte Option. Allerdings nehmen Multithreadingmaßnahmen keinen Einfluss auf die Laufzeitkomplexität. Um diese zu verbessern, würde sich zunächst die Verwendung eines kd-Baums anbieten. Durch einen kd-Baum kann die Suche nach einem zu einer Position nächstliegendem Punkt in $\log(n)$ durchgeführt werden, wenn n die Anzahl an zu durchsuchenden Punkten ist. In der aktuellen Lösung werden dafür innerhalb der Greedyfunktion alle Distanzen ausgerechnet, um die geringste zu finden, was ein lineares Laufzeitverhalten hat. Es sollte jedoch erwähnt werden, dass das

Erstellen des kd-Baums selbst länger dauert als einige wenige Distanzen zu berechnen und zu vergleichen. Es lohnt sich also erst, wenn jedem Zentrum mehrere Datenpunkte zugeordnet werden.¹⁰

3 Fazit

Das in der Aufgabe beschriebene Problem lässt sich als Capacitated Multiple Travelling Salesmen Problem interpretieren. Aufgrund des Laufzeitverhaltens ist eine exakte Lösung nicht realistisch anwendbar. Anstelle dessen wird ein heuristisches Verfahren verwendet, welches auf dem k-Means-Algorithmus und einem Greedy-Verfahren beruht. Auf diese Weise kann die Berechnungszeit in einem Bereich von bis zu mehreren Tausend Pflanzen akzeptabel bleiben. Mit einigen weiteren Veränderungen könnte man diess auf einige wenige Zehntausend Pflanzen ausdehnen.

4 Quellenverzeichnis

- <https://neos-guide.org/case-studies/tra/multiple-traveling-salesman-problem-mtsp/>
- [https://quartz-akz.pages.dev/Travelling-Salesman-Problem-\(TSP\)](https://quartz-akz.pages.dev/Travelling-Salesman-Problem-(TSP))
- <https://antoniocanada.medium.com/introduction-to-the-travelling-salesman-problem-5ace44932cb5>
- <https://www.uni-ulm.de/algo211/>
- <http://ilpubs.stanford.edu:8090/778/1/2006-13.pdf> (S. 3)
- <https://www.cs.ubc.ca/~hutter/previous-earg/EmpAlgReadingGroup/TSP-JohMcg97.pdf> (S. 16ff.)
- https://bwinf.de/fileadmin/wettbewerbe/bundeswettbewerb/44/2_Runde/Aufgaben442.pdf (Aufgabe2, roboter01.txt)
- https://rosettacode.org/wiki/K-d_tree

¹⁰Vgl. https://rosettacode.org/wiki/K-d_tree